Software Re-Engineering

Lecture 8

GraphQL

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

GraphQL is a **language-independent** specification for client-server communication



Why use GraphQL?

- There are a lot of reasons why GraphQL is popular. The first is that it simplifies the communication between frontend and backend developers much less difficult -- instead of frontend developers needing to ask for a new endpoint once their requirements change, they can instead just update their GraphQL query.
- This becomes even more helpful if you have multiple frontends requiring the same backend data. Frontend developers can get exactly the data they need -- no under or over fetching of fields or items.

Why use GraphQL?

- Since frontend developers can request nested data using one query, network requests are also minimized -- for example if you query for a blog post, you can also get that post's comments in that one query instead of doing a second request to get them. This also may reduce the amount of frontend code needed and make that code easier to understand.

Why use GraphQL?

- GraphQL also enforces a typed data schema, so each item's fields will have to match those types. This makes data more consistent and manageable -- instead of having to loop through blog posts and figure out if each title is a string or a boolean, GraphQL will enforce that each title is a string.

- GraphQL has its own type of language that's used to write **schemas**. This is a human-readable schema syntax called Schema Definition Language (SDL).
- The SDL will be the same, no matter what technology you're using — you can use this with any language or framework that you want.

Types

- Types are one of the most important features of GraphQL. Types are custom objects that represent how your API is going to look. For example, if you're building a social media application, your API should have types such as Posts, Users, Likes, Groups.

- **queries** in GraphQL are how you're going to get data. One of the most beautiful things about queries in GraphQL is that you are just going to get the exact data that you want. No more, no less. This has a huge positive impact in our API — no more over-fetching or under-fetching information as we had with REST APIs.

```
type Query {
  users: [User!]!
}
```

the users query will return to us an array of one or more Users. It will not return null, because we put in the ! , which means it's a non-nullable query. It should always return something.

- In GraphQL, **mutations** are the way you're going to modify data on the server and get updated data back. You can think like the CUD (Create, Update, Delete) of REST.

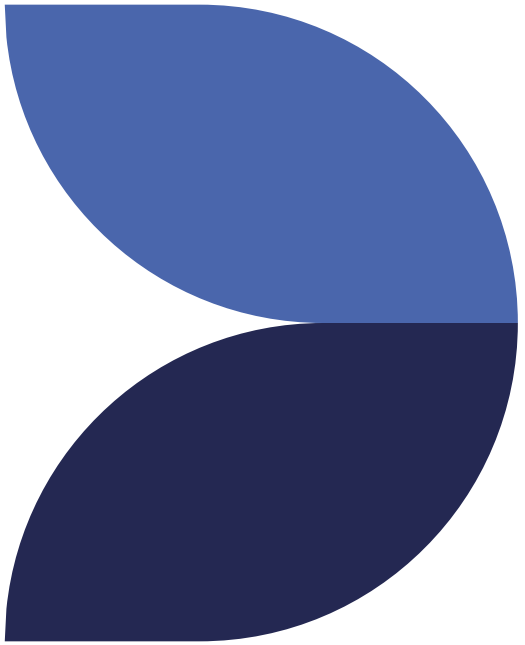
```
type Mutation {
  createUser(id: ID!, name: String!, email: String!, age: Int): User!
  updateUser(id: ID!, name: String, email: String, age: Int): User!
  deleteUser(id: ID!): User!
}
```

As you can see, we're going to have three mutations:

- createUser: we should pass an ID, name, email, and age. It should return a new user to us.

- updateUser: we should pass an ID, and a new name, email, or age. It should return a new user to us.

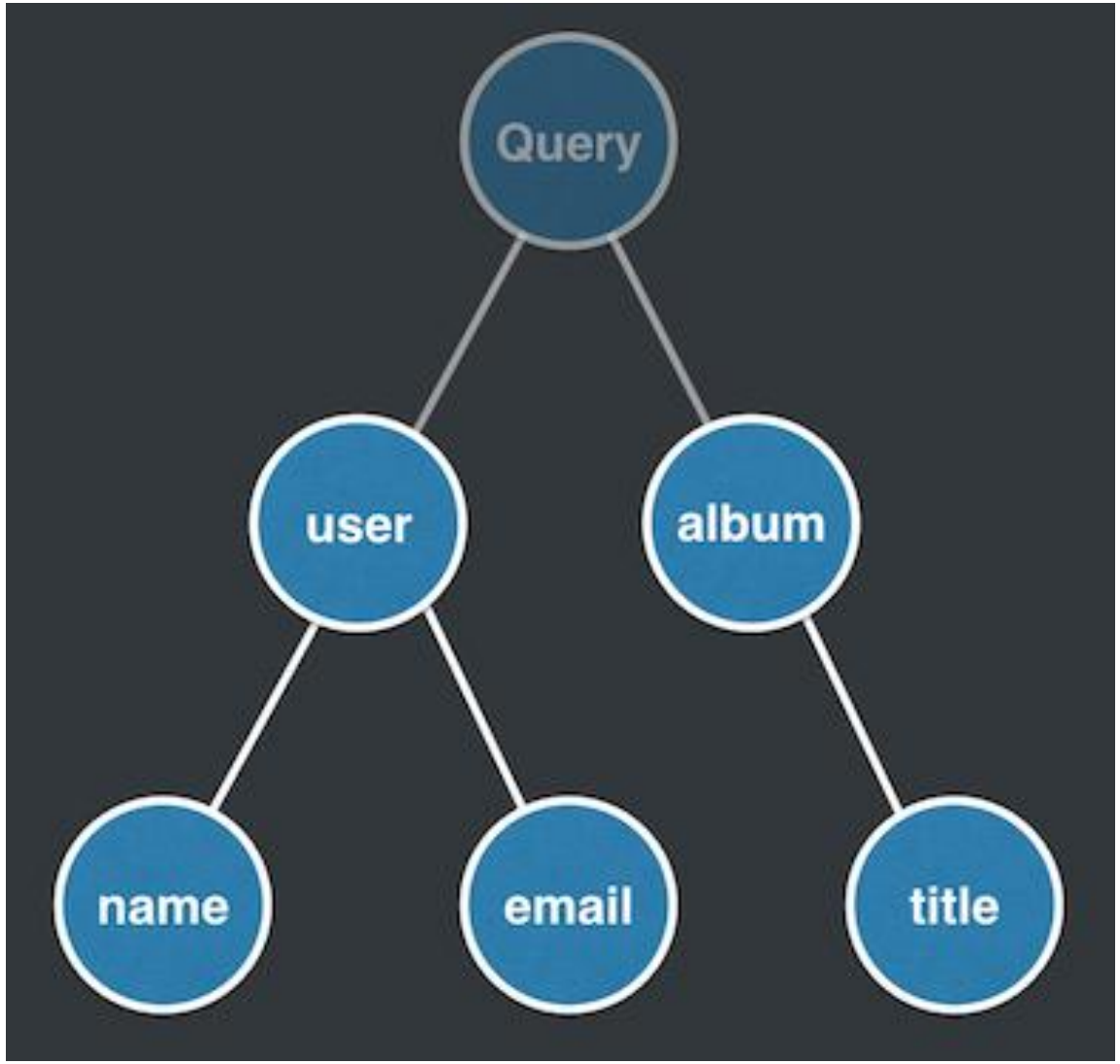
Executing queries

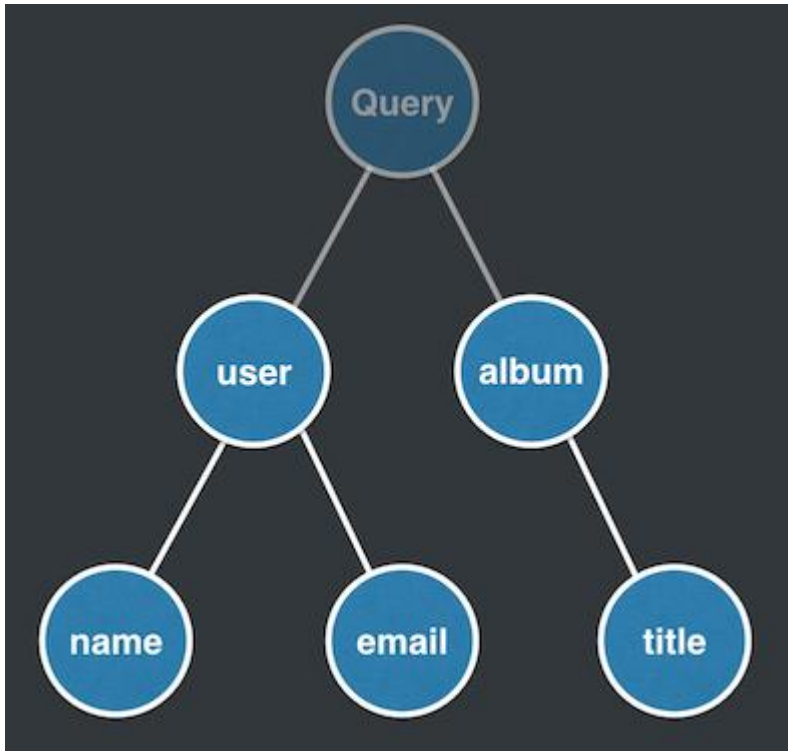


Every GraphQL query goes through three phases. Queries are parsed, validated and executed.

- **Parse** — A query is parsed into an abstract syntax tree (or AST).
- **Validate** — The AST is validated against the schema. Checks for correct query syntax and if the fields exist.
- **Execute** — The runtime walks through the AST, starting from the root of the tree, invokes resolvers, collects up results, and emits JSON.

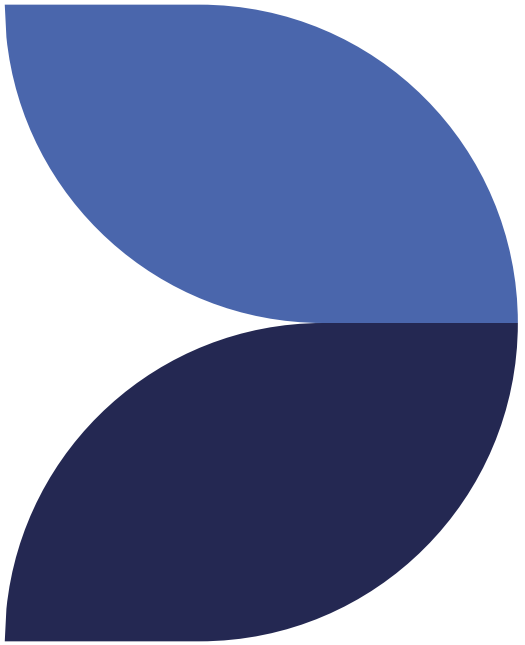
```
query {
  user {
    name
    email
  }
  album {
    title
  }
}
```





- The root Query type is the entry point to the tree and contains our two root fields, ***user*** and ***album.***
- The ***user*** and ***album*** resolvers are *executed in parallel* (which is typical among all runtimes).
- The tree is executed breadth-first, meaning ***user*** must be resolved before its children ***name*** and ***email*** are executed.
- Once all leaf nodes, ***name, email***, ***title***, are resolved, execution is complete.

Resolvers



a **resolver** is a function that handles the process of fetching the data for a **specific query** or **mutation**.

It defines how and where the data comes from when a client sends a request

Why Resolvers Are Needed:

1. **GraphQL only defines the contract, not the implementation**
    1. The GraphQL schema defines **what data is available** and **how it should be structured**.
    2. However, GraphQL itself **doesn’t know where or how** to get the data — that’s where resolvers come in.

2. Resolvers handle business logic

- The resolver defines **how** the data is retrieved (e.g., from a database, an API, or an in-memory object).
- You can add custom business logic, data transformations, and validation within the resolver.

3. Flexibility in data sources

- Data might come from:
  - A **SQL or NoSQL database**
  - A **REST API**
  - A **file**
  - An **in-memory cache**
- The resolver allows you to connect to any of these sources seamlessly.

A GraphQL resolver function typically has the following signature:

(fieldName: (parent, args, context, info) => {})

Parameter	Description
`parent` (or `_`)	Result from the parent resolver (or `null` for root fields).
`args`	Arguments passed from the query or mutation.
`context`	Shared data (like authentication, database connection) available to all resolvers.
`info`	Information about the execution state of the query (like field name, schema details).

Why _ Instead of parent?

- The underscore conventionally indicates that the parameter is not used.
- In the addUser mutation, the resolver doesn't need the parent because it's a root-level mutation, so parent will always be null.
- To avoid a linting warning (e.g., "unused parameter"), developers use _ as a placeholder.

How to Write a Resolver

You first define the schema with the fields you want to resolve

schema.graphql

```
type User {
  id: ID!
  name: String!
  age: Int!
}

type Query {
  users: [User!]!
}

type Mutation {
  addUser(name: String!, age: Int!): User!
}
```

You create resolver functions that match the fields in the schema.

resolvers.js

```
// Sample in-memory data
let users = [
  { id: '1', name: 'Alice', age: 25 },
  { id: '2', name: 'Bob', age: 30 }
];

const resolvers = {
  // Query resolver
  Query: {
    users: () => users,
  },
  // Mutation resolver
  Mutation: {
    addUser: (_, { name, age }) => {
      const newUser = {
        id: String(users.length + 1), // Generate a new ID
        name,
        age
      };
      users.push(newUser); // Add user to the list
      return newUser;
    },
  },
};

module.exports = resolvers;
```

You tie the schema and resolvers together in the GraphQL server.

server.js

```
const { ApolloServer, gql } = require('apollo-server');
const resolvers = require('./resolvers');

// Load schema
const typeDefs = gql`
  type User {
    id: ID!
    name: String!
    age: Int!
  }

  type Query {
    users: [User!]!
  }

  type Mutation {
    addUser(name: String!, age: Int!): User!
  }
`;

const server = new ApolloServer({ typeDefs, resolvers
});

server.listen().then(({ url }) => {
  console.log(`🚀 Server ready at ${url}`);
});
```

Test the Resolver

Start the server using:

```
node server.js
```

Fetch the list of users:

```
query {
  users {
    id
    name
    age
  }
}
```

Test Mutation

Add a new user:

```
mutation {
  addUser(name: "Charlie", age: 28) {
    id
    name
    age
  }
}
```

Confirm Data Updated

Run the query again:

```
query {
  users {
    id
    name
    age
  }
}
```

organized folder structure for a GraphQL project:

```
graphql-demo/
├── src/
│   ├── schema.graphql       # GraphQL schema
│   ├── resolvers.js         # Resolver functions
│   └── server.js            # Apollo Server setup
├── package.json
├── .gitignore
└── README.md
```

Step-by-Step Setup

- GraphQL is a query language, not a standalone runtime or server.
- To use GraphQL, you need to run it on a server — and Node.js is one of the most popular environments for that.
- Most GraphQL server libraries (like Apollo Server and GraphQL-Yoga) are built on top of JavaScript and designed to run in the Node.js environment.

Create a New Project

Create a new project folder:

Install Node.js (if missing)

```
mkdir graphql-demo
cd graphql-demo
```

Initialize a new Node.js project:

```
npm init -y
```

Install the following packages:

```
npm install graphql-yoga
```

GraphQL-Yoga – Lightweight server for quick setup

Create a file server.js with the following code:

```
// server.js
const { createServer } = require('@graphql-yoga/node');

// Sample data
let users = [
  { id: '1', name: 'Alice', age: 25 },
  { id: '2', name: 'Bob', age: 30 }
];

// Define schema and resolvers
const typeDefs = `
  type User {
    id: ID!
    name: String!
    age: Int!
  }

  type Query {
    users: [User!]!
  }

  type Mutation {
    addUser(name: String!, age: Int!): User!
  }
`;

const resolvers = {
  Query: {
    users: () => users
  },
  Mutation: {
    addUser: (_, { name, age }) => {
      const newUser = { id: String(users.length + 1), name, age };
      users.push(newUser);
      return newUser;
    }
  }
};
```

```
// Create server
const server = createServer({
  schema: {
    typeDefs,
    resolvers
  }
});

// Start server
server.start(() => {
  console.log(`🚀 Server running at http://localhost:4000`);
});
```

Update package.json to add a start script:

```
"scripts": {
  "start": "node server.js"
}
```

Start the server using:

```
npm start
```

You should see:

http://localhost:4000

Some more examples...

An example of resolver functions is shown below

```
//resolver function  with no parameters and returning string
greeting:() => {
   return "hello from  TutorialsPoint !!!"
}

//resolver function with no parameters and returning list
students:() => db.students.list()

//resolver function with arguments and returning object
studentById:(root,args,context,info) => {
   return db.students.get(args.id);
}
```

Create a Schema

```
type Query {
   greeting:String
   students:[Student]
   studentById(id:ID!):Student
}

type Student {
   id:ID!
   firstName:String
   lastName:String
   password:String
   collegeId:String
}
```

- The schema file shows that user can query for greeting, students and studentById.
- To retrieve students with specific id, we use data type ID! which shows a non nullable unique identifier field.
- The students field returns an array of students, and greeting returns a simple string value.

resolvers.js

```
const db = require('./db')
const Query = {
   //resolver function for greeting
   greeting:() => {
      return "hello from  TutorialsPoint !!!"
   },

   //resolver function for students returns list
   students:() => db.students.list(),

   //resolver function for studentbyId
   studentById:(root,args,context,info) => {
      //args will contain parameter passed in query
      return db.students.get(args.id);
   }
}
module.exports = {Query}
```

write a resolver for the Student type. Our schema file looks like this:

```
type Query {
  student(id: String!): Student
}

type Course {
  id: String!
  title: String
}

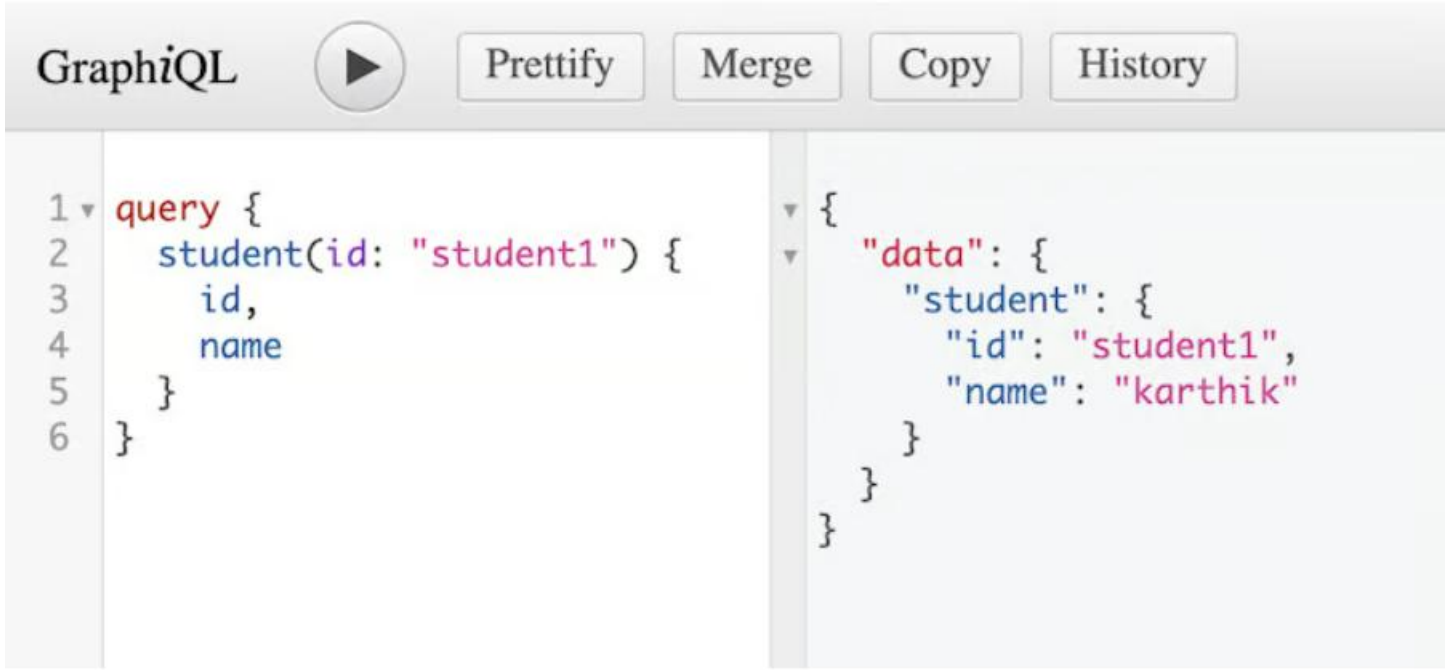
type Student {
  id: String!
  name: String
  courses: [Course]
}
```

```
var students = {
  'student1': {
    id: 'student1',
    name: 'karthik',
    courses: ['math101', 'geography201']
  },
  'student2': {
    id: 'student2',
    name: 'john',
    courses: ['physics201', 'chemistry103']
  },
};
var courses = {
  'math101': {
    id: 'math101',
    title: 'Intro to algebra',
  },
  'geography201': {
    id: 'geography201',
    title: 'Intro to maps',
  },
  'physics201': {
    id: 'physics201',
    title: 'Intro to physics',
  },
  'chemistry103': {
    id: 'chemistry103',
    title: 'Intro to organic chemistry',
  },
};
```

```
const resolvers = {
  Query: {
    student: (root, args, context, info) => {
      return students[args['id']]
    }
  }
}

module.exports = resolvers
```

The resolver for student takes an id in its args and returns the corresponding student from the students object:

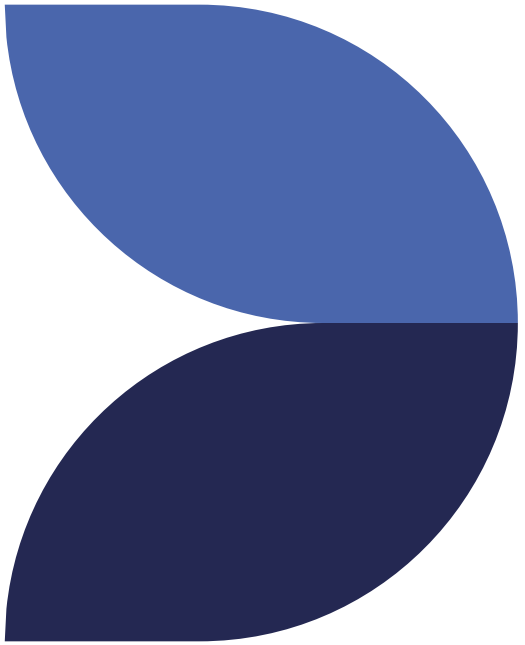


root

Every graphQL type has a default resolver. When you don’t write a resolver for a type or a field, GraphQL automatically looks into the root for a property with the same name as the field or a type. The default resolver will look something like this:

```
const resolvers = {
  Query: {
    student: (root, args, context, info) => {
      return students[args['id']]
    },
  },
  Student: {
    name: (root, args, context, info) => {
      return root.name;
    }
  }
}
```

Issues with resolvers



Overfetching

write a resolver for "courses" in the student type and fetch the courses along with the student query

```
const resolvers = {
  Query: {
    student: (root, args, context, info) => {
      const studentCourses =
students[args['id']]['courses'].map(id => {return
courses[id]})
      return {
        ...students[args['id']],
        "courses": studentCourses
      }
    },
  },
  Student: {
    name: (root, args, context, info) => {
      return root.name;
    }
  }
}
```

Problem 1: What happens if we write a query that asks only for the student id and name?

We would still be unnecessarily doing the operation. In a real-world scenario, this could even be an expensive API call. But when GraphQL resolves the query, it would drop the extra data on the floor.
On the surface, it still seems like we are getting only what we asked for. But behind the scenes, we have forced our server to overfetch because of the way we designed our resolver.

```
const resolvers = {
  Query: {
    student: (root, args, context, info) => {
      const studentCourses =
students[args['id']]['courses'].map(id => {return
courses[id]})
      return {
        ...students[args['id']],
        "courses": studentCourses
      }
    },
  },
  Student: {
    name: (root, args, context, info) => {
      return root.name;
    }
  }
}
```

Solution: Move the courses resolver to the courses field.

```
const resolvers = {
  Query: {
    student: (root, args, context, info) => {
      return students[args['id']];
    },
  },
  Student: {
    name: (root, args, context, info) => {
      return root.name;
    },
    courses: (root, args, context, info) => {
      return root.courses.map(id => courses[id]);
    }
  }
}
```

leveraging the root argument

- That's why it's useful to visualize the query as a tree and understand the root node and how you can use it.
- Now, if we just query for id and name, we are not at risk of resolving the courses unnecessarily.

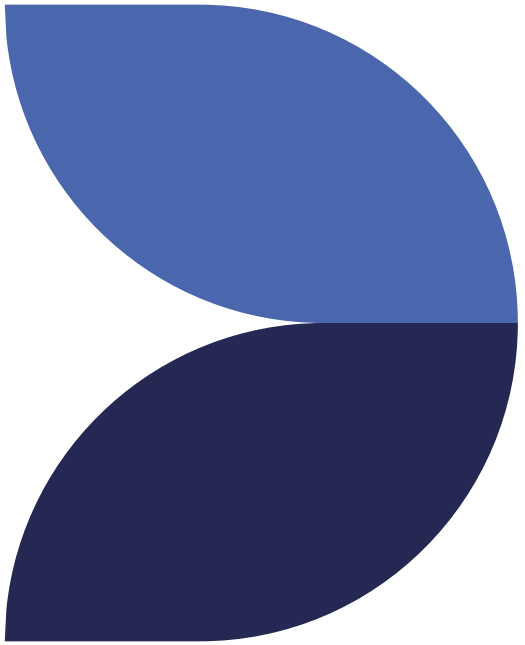
Problem 2: What happens if we write a query to fetch only for the courses?

- You might think it is going to resolve only the courses resolver. But, again, think about the tree — what gets resolved first? The student resolver!
- The student node is the parent of the courses node, and GraphQL resolves in a breadth-first search fashion, which means the student node gets resolved before the courses node so that the courses node's root argument is populated.

  Again, we run into overfetching. What to do now?
- Let's move the student resolver down to its fields and resolve the fields separately.

```
const resolvers = {
  Query: {
    student: (root, args, context, info) => {
      return args['id'];
    },
  },
  Student: {
    id: (root, args, context, info) => {
      return students[root]['id'];
    },
    name: (root, args, context, info) => {
      return students[root]['name'];
    },
    courses: (root, args, context, info) => {
      return students[root]['courses'].map(id => courses[id]);
    }
  }
}
```

Practice, ToDo task



How can I create a simple React application that communicates with a GraphQL API to manage a list of books using Apollo Client?

Setup Your Project:
First, make sure you have Node.js installed on your system. If not, you can download it from https://nodejs.org/. Then, create a new directory for your project and initialize it with npm or yarn:

```
mkdir graphql-book-api
cd graphql-book-api
npm init -y
```

Then, create a new React application using Create React App:

```
npx create-react-app graphql-book-app
cd graphql-book-app
```

Install Dependencies:
You'll need to install the necessary dependencies, including @apollo/client to interact with your GraphQL server. Run the following command to install these packages:

npm install @apollo/client graphql

Create a GraphQL Client:

Create a file named apollo.js to set up your GraphQL client. You'll need to configure the client to connect to your GraphQL server:

```
import { ApolloClient, InMemoryCache, HttpLink } from '@apollo/client';

const client = new ApolloClient({
  cache: new InMemoryCache(),
  link: new HttpLink({
    uri: 'YOUR_GRAPHQL_SERVER_URL', // Replace with your GraphQL server's URL
  }),
});

export default client;
```

Define Your React Components:

Create your React components for displaying the list of books. You can use the useQuery hook from Apollo Client to fetch data from your GraphQL API. Here's an example component:

```
import React from 'react';
import { useQuery } from '@apollo/client';
import gql from 'graphql-tag';

const GET_BOOKS = gql`
  query GetBooks {
    books {
      title
      author
    }
  }
`;

function BookList() {
  const { loading, error, data } = useQuery(GET_BOOKS);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return (
    <div>
      <h2>Book List</h2>
      <ul>
        {data.books.map((book, index) => (
          <li key={index}>
            <strong>Title:</strong> {book.title},
<strong>Author:</strong> {book.author}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default BookList;
```

Integrate Components into App:

Use your BookList component (or any other components you create) within your main App.js file.

Run Your React App:

Start your React application using:

npm start
Your React app will run at http://localhost:3000.

Connect to GraphQL API:

Ensure that your GraphQL server is running and accessible at the URL you specified in the apollo.js configuration.

You've now created a simple React application that communicates with a GraphQL API to manage a list of books. You can expand on this example by adding features like creating, updating, or deleting books using GraphQL mutations.